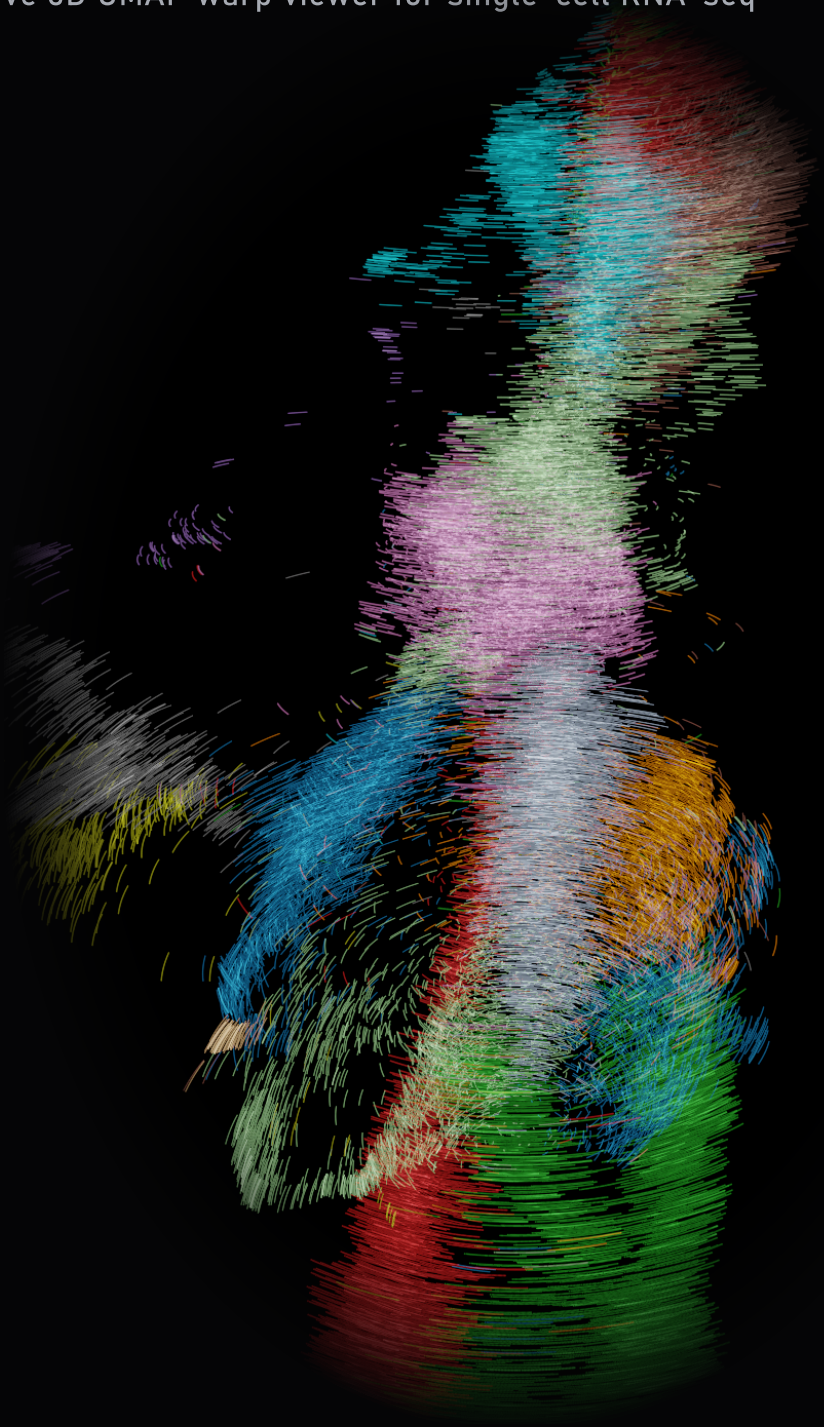
USER MANUAL

scAnimator

Interactive 3D UMAP warp viewer for single-cell RNA-seq



WebGL · Three.js · Python · R

Version 1.5, November 2026

Live demo

gsnautilus.github.io/scAnimator

CONTENTS

Every picture of the viewer in this manual was taken from the viewer itself, by `manual/capture.py`. The small dataset is the public PBMC 3k demo you can open in the live demo. The larger one is a ~39,000-cell neural dataset; its pictures are here, but its data is not part of the project.

1	**Quick start**	from the live demo to your first video	3
2	**The viewer at a glance**	what is where on screen	4
3	**What the animation shows**	n_neighbors, keyframes and the warp	5
4	**Colour, filter, normalize**	metadata, legend, gradients	6
5	**Gene expression**	colormap, size and intensity overlays	8
6	**Render settings**	brightness, bloom, persistence, depth of field, tracers	9
7	**Playback and camera**	the timeline, morph, 360, mouse	10
8	**Recording video**	frame-exact PNG sequences, and MP4 from them	11
9	**Storyboards**	scripted, captioned tours: present or render	12
10	**Experimental**	audio reactivity, fractal reflection	14
11	**Bring your own data**	R or Python to WebGL: build_viewer.py	15
12	**Troubleshooting**	and the file formats	17

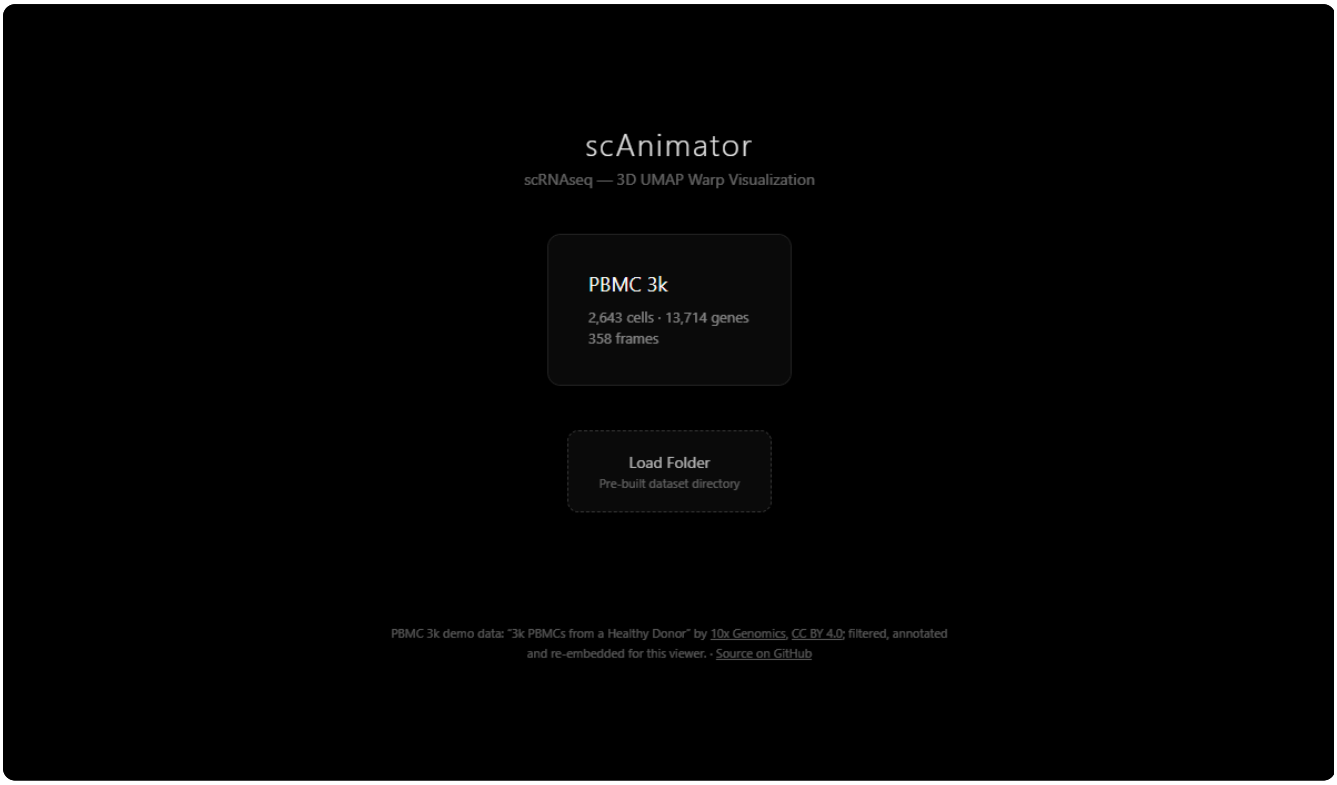
START HERE

1 Quick start

scAnimator runs in a web browser. Nothing to install: open the live demo and the PBMC 3k dataset starts moving.

---

1. **Open the demo.** Go to gsnautilus.github.io/scAnimator in a current Chrome, Edge or Firefox (WebGL2 is needed). The landing page lists the datasets; click `PBMC 3k`.



The landing page. One card per dataset, with its size. `Load Folder` opens a dataset you built yourself (p. 15).

2. **Watch it warp.** The cells are already moving: the embedding sweeps from *n_neighbors* 5 to 150 and back. The label at the top says where it is.
3. **Turn it around.** Drag to rotate, scroll to zoom. `360` makes the camera circle on its own.
4. **Colour it.** Open `Color By` in the panel on the left and pick **Cell Type**. Click a name in the legend to hide those cells.
5. **Look up a gene.** Open `Gene Expression`, type LYZ and press Enter: monocytes light up.
6. **Make it glow.** In `Render Settings`, raise `Point Size` and `Bloom Strength`.
7. **Record.** Set a duration, press `Record`, and the viewer renders one exact loop to a ZIP of PNG frames (p. 11).

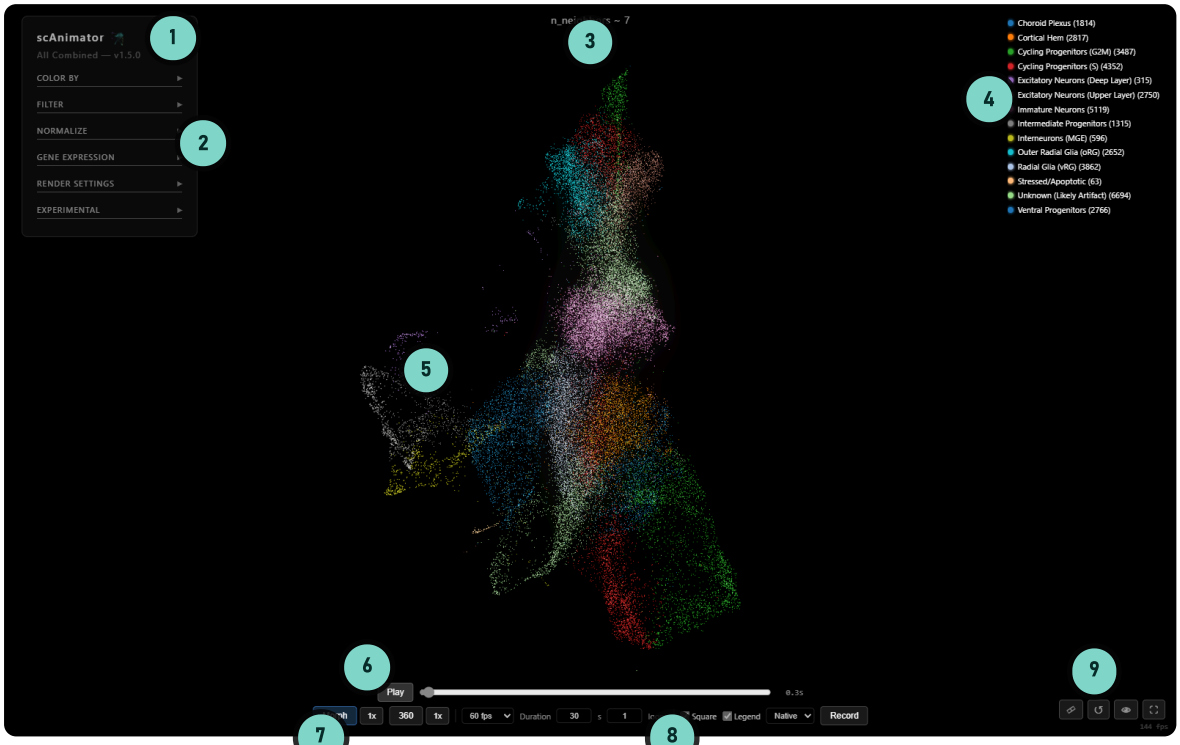
GOOD TO KNOW

- Everything happens in your browser. A dataset you load from a folder is never uploaded anywhere.
- Genes are fetched one at a time as you ask for them, so even a large dataset opens quickly.
- ↺ (bottom right) puts the camera back if you get lost.

ORIENTATION

2 The viewer at a glance

The cloud fills the window. Everything you can change sits around it, and all of it hides with one button when you want the picture alone.



1. **Header.** The dataset's name and the version. The folder button goes back to the landing page.
2. **Panel.** Six groups that open and close: Color By, Filter, Normalize, Gene Expression, Render Settings, Experimental.
3. **n_neighbors label.** Which embedding you are looking at (or between).
4. **Legend.** Categories with cell counts. Click one to hide or show it.
5. **The cloud.** One point per cell. Drag to rotate, scroll to zoom.
6. **Timeline.** Play/Pause and the playhead: drag it to scrub.
7. **Motion.** `Morph` and `360`, each with a speed.
8. **Recording.** Frame rate, duration, loops, square crop, legend, resolution, Record.
9. **Corner buttons.** Storyboard, reset view, hide UI, fullscreen; the frame rate below them.

PANEL GROUPS

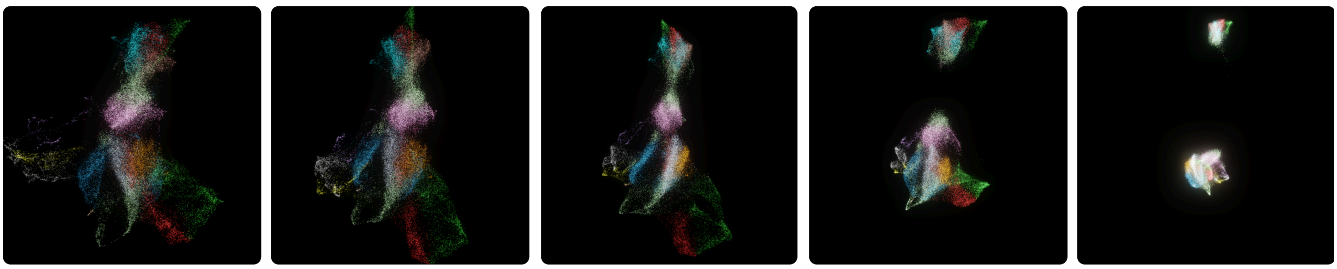
GROUP	WHAT IT DOES	PAGE
Color By	colour cells by metadata, a gene, a solid colour or a moving gradient	p. 6
Filter	show or hide cells by any metadata column	p. 6
Normalize	equal cell counts per group, so big groups don't dominate	p. 6
Gene Expression	overlay one gene as colour, size or brightness	p. 8
Render Settings	points, glow and motion effects	p. 9
Experimental	audio reactivity and fractal reflection	p. 14

THE IDEA

3 What the animation shows

A UMAP plot is one answer to a question with a dial on it. scAnimator turns the dial and lets you watch.

UMAP places cells so that each one stays near its *n_neighbors* most similar cells. With a small value it only cares about very close neighbours, so fine, local structure shows and the map can break into many islands. With a large value it balances each cell against more of the dataset, so the broad, global arrangement shows and small groups merge. Neither picture is the true one. Watching one become the other shows which clusters are robust and which only appear at one setting.



The same ~39,000 neural cells at five *n_neighbors* values, same camera. At 150 one population splits off as a separate island.

How the frames are made

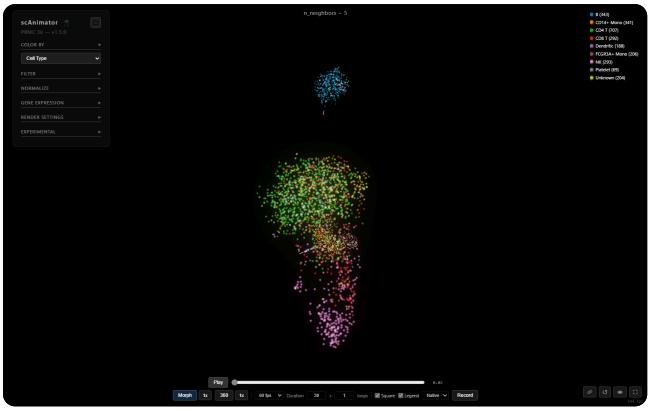
1. **Keyframes.** `build_viewer.py` runs a 3D UMAP at a set of *n_neighbors* values, for example 11 values from 5 to 150 (`--sweep 5-150-11`), all from the same PCA input.
2. **Alignment.** Each UMAP is free to come out rotated or mirrored. Every keyframe is Procrustes-aligned to the one before it (rotation, reflection and scale only), so whatever still moves is a real change in structure.
3. **In-between frames.** A cubic spline through the aligned keyframes gives every cell a smooth path (180 frames by default). Playback runs up the sweep and back down, so it loops.
4. **In the browser.** All frames load at once as one binary file; the viewer blends between neighbouring frames, so motion stays smooth at any speed.

> READING IT CAREFULLY
>
> - The in-between frames are interpolations for the eye, not UMAP runs. Only the keyframes are real embeddings, which is why the label reads `n_neighbors ~ 30`: between keyframes the value is approximate.
> - Distances between far-apart islands mean little in any UMAP. What a cell keeps as neighbours through the sweep means more.

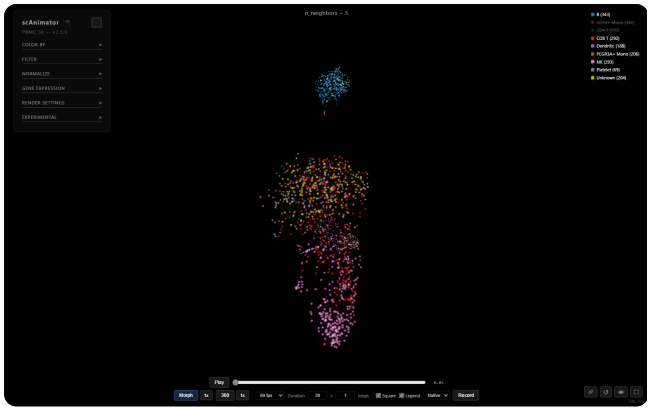
METADATA

4 Colour, filter, normalize

Every categorical column the dataset was built with can colour the cells, filter them, or even out their numbers.



Color By: Cell Type. Each category gets a colour; the legend lists them with counts.



Legend toggles. CD4 T and CD14+ Mono clicked off: their names dim and their cells disappear.

Color By

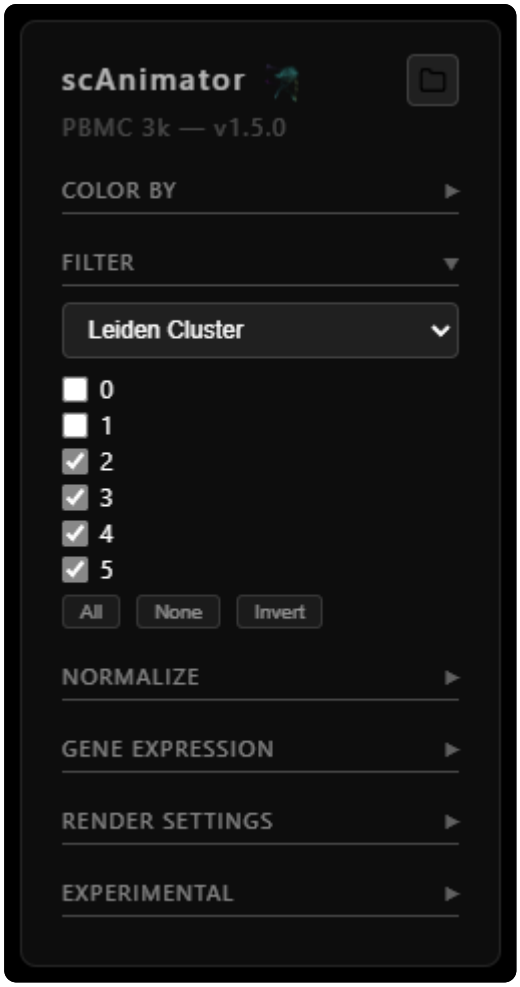
CHOICE	COLOURS
a metadata column	one colour per category (cell type, cluster, sample…), with a legend
Gene Expression	the gene loaded in the Gene Expression group, through its colormap
Solid Color	every cell one colour, picked with the colour swatch that appears
Gradient	every cell cycles through a colormap in time with the timeline
Gradient (Cluster)	the same, but each cluster starts at its own place in the colormap, so clusters pulse out of step

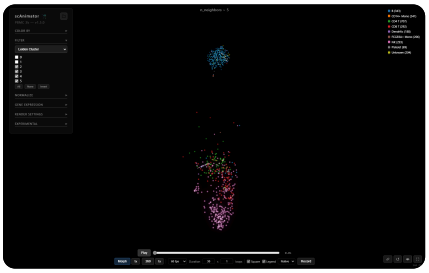
Filter

Pick a column, then tick the categories to show. `All`, `None` and `Invert` work on the whole list. Filters on different columns combine: a cell shows only if every column lets it. The filter column does not have to be the colour column: here the cells are coloured by cell type and Leiden clusters 0 and 1 are hidden.

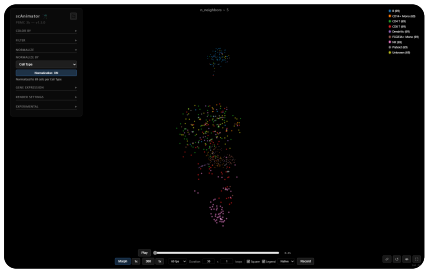
Normalize

Pick a column and press `Normalize Cell Counts`. Every group is cut down to the size of the smallest by a fixed random pick (the same cells every time), so a large population no longer hides a small one. Legend counts follow. Press it again to see every cell.

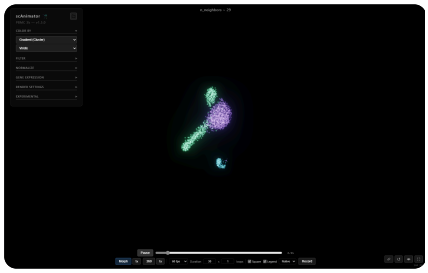




Filter. Leiden 0 and 1 unticked, colours still by cell type.



Normalize by Cell Type. 69 cells of each type: the size of the smallest group.



Gradient (Cluster). Clusters cycle through the colormap out of step.

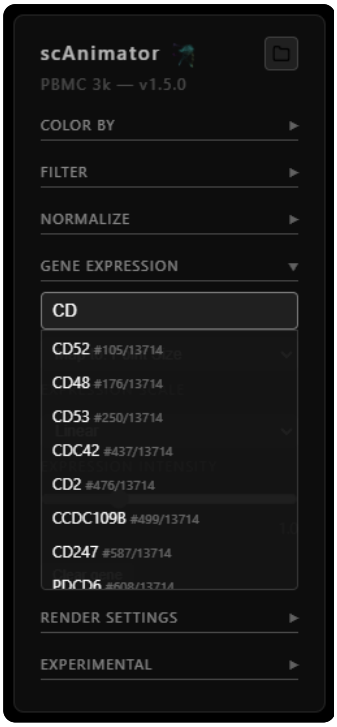
EXPRESSION

5 Gene expression

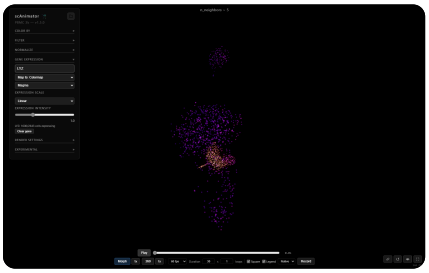
Type a gene and see where it is expressed, as colour, as point size or as brightness, while the embedding keeps moving.

1. **Type a gene symbol.** Suggestions appear as you type, most-expressed genes first, with their rank (#42/13714). Use ↑ ↓ and Enter, or click one.
2. **Choose how to show it.** `Map to` : Point Size, Colormap or Intensity.
3. **Adjust the scale.** Linear, Log or Quantile, and `Expression Intensity` to push faint signal up or strong signal down.

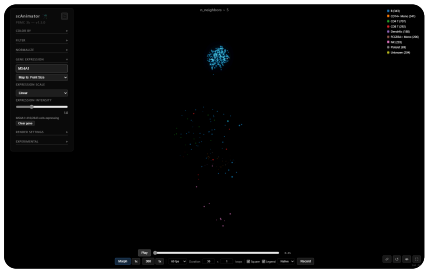
Under the controls the viewer says how many cells express the gene (`LYZ: 1600/2643 cells expressing`). `Clear gene` goes back to the plain colours. Expression is stored per gene relative to that gene's own maximum: compare genes by pattern, not by brightness.



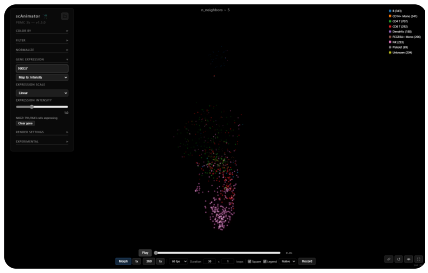
Autocomplete, ranked by total expression.



Colormap. LYZ in magma: the monocytes are bright.



Point Size. MS4A1 (CD20) keeps the cell-type colours and swells the B cells.



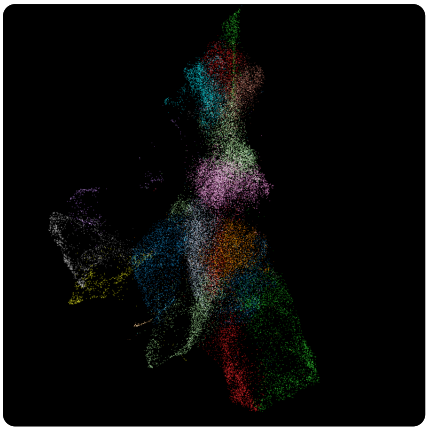
Intensity. NKG7 dims everything that doesn't express it: NK and CD8 T remain.

CONTROL	OPTIONS	WHAT IT DOES
Map to	Point Size · Colormap · Intensity	Size and Intensity keep the Color By colours and scale size or brightness by expression. Colormap replaces the colours and shows a colour bar.
Colormap	Viridis, Magma, Plasma, Inferno, Rainbow, Turbo, Coolwarm	the colours for Colormap mapping
Expression Scale	Linear · Log · Quantile	Log lifts low values; Quantile spreads cells evenly over the range, good for genes with a few extreme cells
Expression Intensity	0.2 – 3.0 (1.0)	a gamma on the scaled values: above 1 brightens weak expression

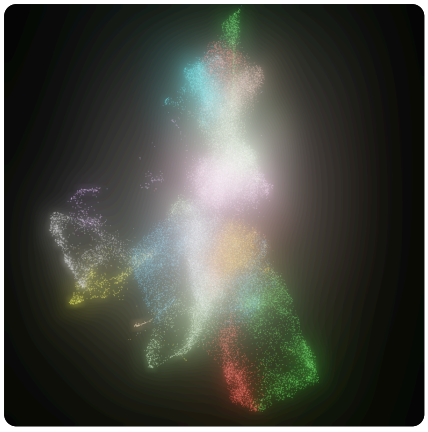
LOOK

6 Render settings

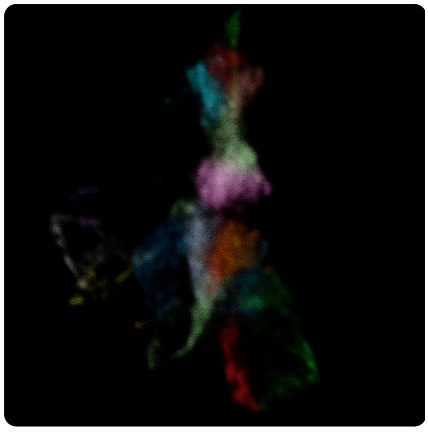
Each cell is drawn as a small light: points add up where they overlap, so dense regions glow. These settings shape that light.



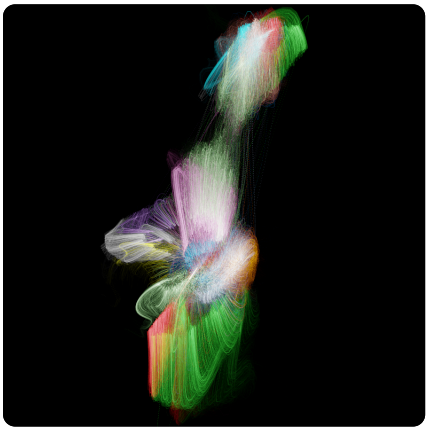
Plain. Point size 16, no effects.



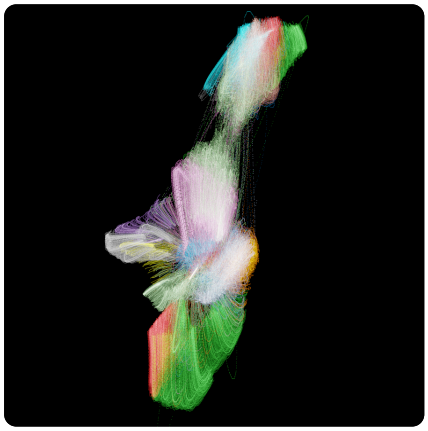
Bloom 1.4. Bright regions bleed light.



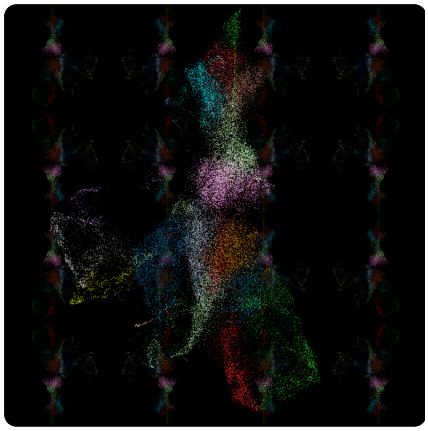
Depth of field. Far cells blur, near ones stay sharp.



Persistence. Old frames fade slowly: motion leaves streaks.



Tracers. Each cell draws its path over the last frames.



Fractal reflection (Experimental).

SLIDER	RANGE (START)	WHAT IT DOES
Point Intensity	0.05 – 1.0 (1.0)	brightness of each point. Lower it for dense datasets so overlaps don't burn out to white.
Bloom Strength	0 – 4 (0)	glow around bright regions (Three.js UnrealBloom)
Point Size	2 – 40 (2)	size of each point. Points are drawn in perspective, so they grow as you zoom in.
Persistence	Off, 1 – 100	how slowly old frames fade: a trail of light behind moving cells
Depth of Field	Off, 1 – 100	near cells stay sharp, farther ones blur more and more: a sense of depth
Tracer Length	Off, 1 – 180 frames	lines along each cell's recent path, crisp rather than fading

A STARTING LOOK

- Small dataset (a few thousand cells): Point Size 30–40, Bloom 0.5, Intensity 0.8.
- Large dataset (tens of thousands): Point Size 12–18, Intensity 0.5–0.7, Bloom 0.3. More cells need less light each.
- Persistence and tracers only show while something moves: turn on Morph or 360.

TIME AND SPACE

7 Playback and camera

One timeline drives everything that moves, so a loop always comes back to where it started. That is what makes recordings loop seamlessly.



The control bar. Top row: playback. Bottom row: what moves, then how it is recorded. The corner buttons sit at the bottom right.

The timeline

The playhead runs for **Duration** seconds (30 by default) and starts again. Within that time each moving thing completes **loops** cycles. Drag the playhead to scrub; **Pause** holds the current frame, and you can still rotate and change anything.

CONTROL	OPTIONS	WHAT IT DOES
Play / Pause		runs or holds the timeline
Playhead	0 – Duration	drag to any moment; the n_neighbors label follows
Morph	on / off	the n_neighbors sweep. Off: the embedding holds still while everything else carries on.
Morph speed	0.1, 0.25, 0.5, 1, 2, 4×	sweeps per loop. Click to step through.
360	on / off	circles the camera around the view's current up axis, at the current zoom
360 speed	0.1 – 4×	turns per loop
Duration	1 – 600 s	length of the timeline (and of a recording)
loops	1 – 99	how many times Morph and 360 cycle within one Duration

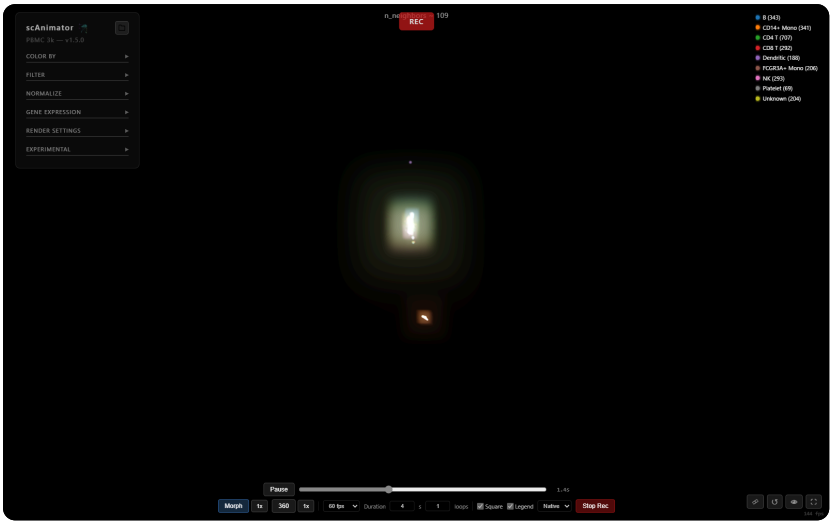
Mouse and keys

DO	TO
Drag	rotate around the centre of the cloud
Scroll	zoom in and out
↺ **Reset view**	camera back to the start (and leaves a storyboard)
👁 **Hide UI**	hide panel, legend, labels and controls for a clean picture
⛶ **Fullscreen**	the browser's full screen; Esc leaves it
In a storyboard	→ ↓ Space PgDn next step; ← ↑ PgUp back; Home first step (presentation clickers work)

EXPORT

8 Recording video

Recording does not capture the screen. It steps through the timeline frame by frame, so every frame is exact at any resolution, however slow the computer.



Recording. REC shows at the top, the button reads `Stop Rec`. The legend in the corner is drawn into the frames when `Legend` is ticked.

1. **Set up the view.** Colours, gene, render settings, camera angle. Turn on Morph and/or 360.
2. **Choose the loop.** `Duration` is the length of the video; `loops` how many sweeps or turns fit in it. Pick the frame rate.
3. **Choose the frame.** `Square` crops to a centred square; `Native` records at the window's size, `2K` and `4K` render square frames of 2160 or 3840 pixels.
4. **Press Record.** The timeline restarts and renders Duration × fps frames. It stops by itself at the end of the loop (or press `Stop Rec`) and downloads `umap_warp_frames_60fps.zip`: numbered PNG frames and `fps.txt`.

Making an MP4

Unzip the frames and turn them into a video with ffmpeg (PowerShell or any shell, in the unzipped folder):

```
ffmpeg -framerate 60 -i frame_%05d.png -c:v libx264 -crf 16 -pix_fmt yuv420p umap_warp.mp4
```

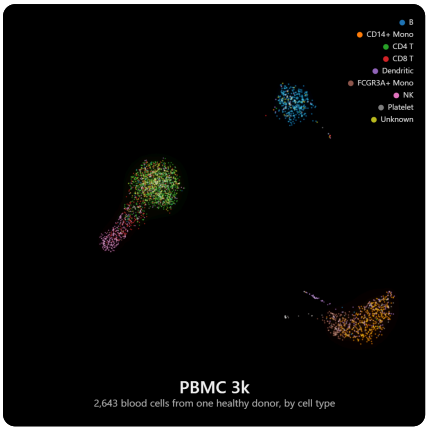
Use the number in `fps.txt` for `-framerate`. Because the loop ends where it started, the MP4 loops seamlessly too. Frames are written straight into the ZIP (ZIP64), so even long 4K recordings don't run out of memory.

SETTING	OPTIONS	NOTES
fps	24, 30, 60, 120	frames per second of the result, not of the screen
Duration · loops	1–600 s · 1–99	shared with playback (p. 10)
Square	on (default) / off	centred square crop, for social media and slides
Legend	on (default) / off	draws the legend (or gene colour bar) into the frames
Resolution	Native, 2K, 4K	2K and 4K render off-screen at that size, independent of the window

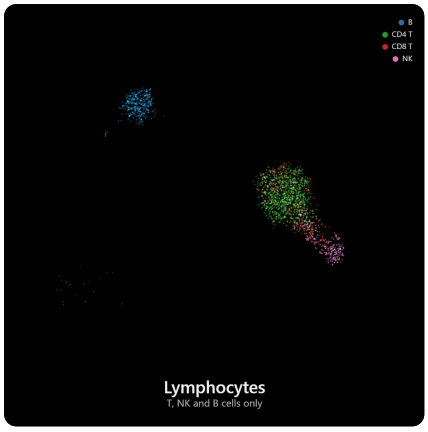
TELL A STORY

9 Storyboards

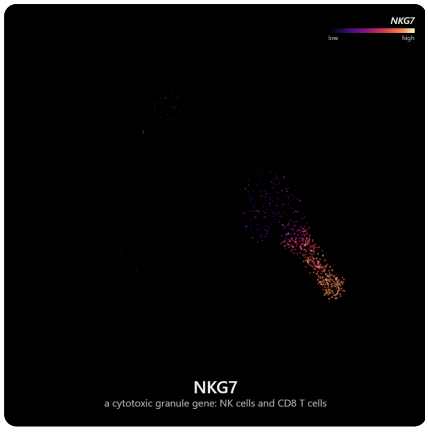
A storyboard is a short JSON file: a list of views, each with a caption. Present it live with a clicker, or render it to MP4 clips.



1 overview. By cell type, with caption and legend.



3 lymphocytes. A filter fades the other cells out.



4 nkg7. Coloured by a gene, with its colour bar.

Each step is a **view**: what to colour by, which cells to show, which *n_neighbors* frame to hold. Between steps the viewer **transitions**: cells fade in and out, colours cross-fade, and if the n_neighbors changes the embedding morphs. The camera keeps turning throughout. A step can also be a **slide**, a full-frame card of text and an image that fades over the cloud.

```
{
  "dataset": "pbmc3k",
  "render": { "point_size": 40, "brightness": 0.8, "bloom": 0.5 },
  "camera": { "zoom": 1.2, "elevation": 15 },
  "steps": [
    { "name": "overview", "color_by": "cell_type", "n_neighbors": 5,
      "caption": "PBMC 3k", "subcaption": "2,643 blood cells, by cell type" },
    { "name": "lymphocytes", "filter": { "cell_type": ["CD4 T", "CD8 T", "NK", "B"] },
      "caption": "Lymphocytes" },
    { "name": "nkg7", "color_by": "gene:NKG7", "colormap": "magma", "caption": "NKG7" }
  ]
}
```

The full example is `examples/pbmc3k_tour.json` in the repository.

Present it

Load it with the film-strip button, by dropping the `.json` (and any slide images) onto the viewer, or with `viewer.html?dataset=pbmc3k&storyboard=<url>`. Each step holds, turning, until you press →. ↺ leaves the storyboard and puts your own settings back.

Render it

```
python scripts/render_storyboard.py examples/pbmc3k_tour.json            # MP4 clips + _full.mp4
python scripts/render_storyboard.py examples/pbmc3k_tour.json --stills   # one frame per step: a quick check
```

The script drives the viewer in Edge (Playwright) frame by frame and encodes with ffmpeg: per step a transition clip and a seamlessly looping hold clip. `--draft` renders half size.

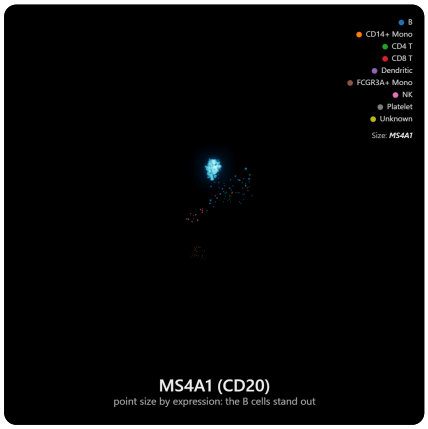
Storyboard reference

TOP-LEVEL FIELD	DEFAULT	MEANING
`dataset`		dataset id (the folder name in `datasets/`), for rendering
`size, fps`	1080, 30	rendered frame size (square) and rate
`turn_seconds`	24	one full camera turn; a step holds for whole turns
`transition_seconds`	2	default transition between steps
`slide_seconds`	6	default hold for a slide
`camera`	auto	`distance`, `target` (auto: fit 98% of cells), `zoom`, `elevation`, `start_angle`
`render`	{}	`point_size`, `brightness`, `bloom`; `dof`, `trails`, `tracers`, `fractal` (off unless named)
`colors`	{}	palette overrides: `{"cell_type": {"B": "#3fa7ff"}}`
`style`		caption, subcaption, legend and slide font sizes; `legend_side`; `slide_background`

STEP FIELD	INHERITED	MEANING
`name`	no	used for clip file names and `--only`
`caption, subcaption`	no	text at the bottom of the frame
`color_by`	yes	a metadata column, or `gene:<symbol>`
`filter`	yes	`{column: [values shown]}`; a step's filter replaces the previous one; `{}` shows all
`n_neighbors`	yes	the frame to hold (nearest keyframe); a change morphs
`gene, gene_mapping`	mapping only	scale size or intensity of the categorical colours by a gene
`colormap, gene_scale, gene_intensity`	yes	as in the Gene Expression panel
`legend, legend_counts`	legend	show the legend; with counts
`turns`	no	hold length in camera turns (default 1)
`transition`	no	seconds of transition into this step
`slide`	no	`title`, `body` (text or a list of bullets), `image`, `layout` (stack, left, right), `background`; hold with `seconds`



A slide. Title and bullets over the cloud, which keeps turning underneath.



gene + gene_mapping. Cell-type colours, size by MS4A1.

10

FOR PERFORMANCE

Experimental

Two effects made for projection and live visuals rather than analysis.

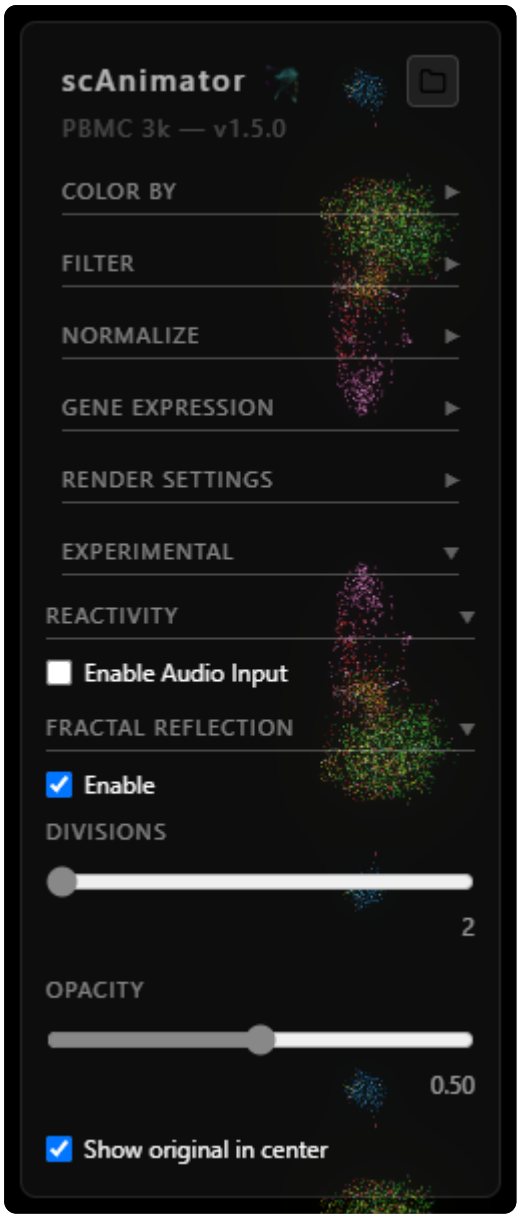
Audio reactivity

Tick **Enable Audio Input** and allow the microphone (or pick another input device, such as a loopback of your music). The sound is split into three bands, each with a level meter, and each band drives one property of every cell.

BAND	RANGE	DRIVES (START)
Bass	20 – 250 Hz	Point Size
Mids	250 Hz – 4 kHz	Intensity
Highs	4 – 20 kHz	Colormap

Each band can map to Point Size, Intensity, Colormap or **Displacement** (cells jitter out of place), and has a Scale curve (linear, log to boost quiet sound, exponential to boost peaks), Sensitivity, Strength, Smoothing and Spread (how differently cells respond).

- **Peak Hold** keeps each band at its last peak and lets it fall slowly.
- **Cluster Mapping** gives each cluster of a chosen column one band, in turn: clusters answer different parts of the music.
- **Expression Weight** scales the response by the loaded gene: expressing cells react most.



Fractal reflection

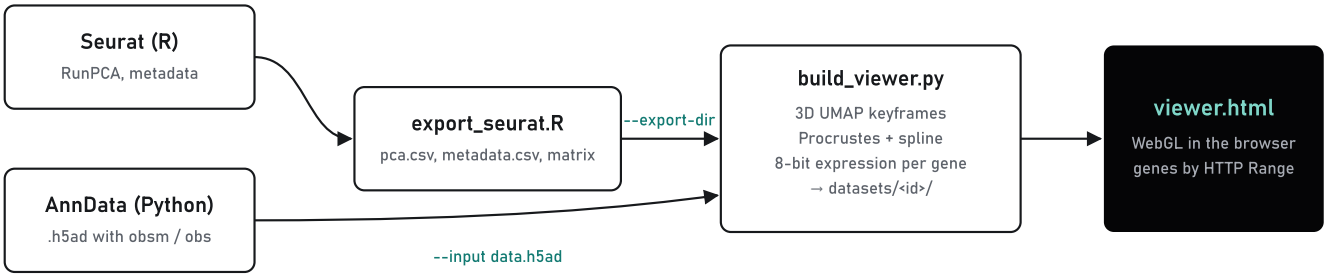
Repeats the picture as a kaleidoscope of smaller copies around the centre. **Divisions** (2–8) sets how many, **Opacity** how strong the copies are, and **Show original in center** keeps the full-size cloud on top (picture on p. 9).

YOUR OWN DATASET

11

Bring your own data

Any dataset with a PCA (or another embedding) and cell metadata can be animated. One Python script turns it into the files the viewer reads.



From Python (AnnData)

```
pip install numpy pandas scipy scanpy umap-learn h5py
python scripts/build_viewer.py --input my_data.h5ad --output-dir datasets/my_data --name "My data" --
sweep 5-150-11
python scripts/update_dataset_index.py
```

The UMAP input is `obsm["X_pca"]` unless you name another embedding (`--embedding-key X_scvi`); with no suitable embedding, PCA is computed. The colour column is found automatically (`cell_type`, `leiden`, `louvain`…) or set with `--cell-type-col`. Every categorical `obs` column becomes a Color By and Filter option unless you list them with `--metadata-cols`.

From R (Seurat)

```
source("scripts/export_seurat.R")
export_for_scanimator(obj, "export", cell_type = "cell_type", columns = c("seurat_clusters",
"orig.ident"))
```

```
python scripts/build_viewer.py --export-dir export --output-dir datasets/my_data --name "My data" --
sweep 5-150-11 --metadata-cols cell_type,seurat_clusters,orig.ident
```

`export_seurat.R` writes the PCA, the metadata and the normalized expression matrix (sparse, as binary files) that `--export-dir` reads. It works with Seurat v4 and v5.

Options

OPTION	DEFAULT	WHAT IT DOES
`--sweep a-b-n`	none	n keyframes from n_neighbors a to b. Without it: one static UMAP (fast)
`--n-interp-frames`	180	frames between the first and last keyframe
`--umap-neighbors`	15	n_neighbors of the static UMAP
`--embedding-key`	X_pca	the `obsm` key used as UMAP input
`--no-pca`		run UMAP on expression instead of computing a PCA (slow)

`--cell-type-col`	auto	the default colouring column
`--metadata-cols`	all categorical	comma-separated columns to include
`--hvg-only`		store only highly variable genes: a much smaller `expression.bin`
`--skip-umap / --skip-expression`		reuse what an earlier run wrote
`--name`	folder name	display name on the landing page and in the panel

Above 200,000 cells the script reads the `.h5ad` in backed mode and streams expression through `h5py`, so atlases of a million cells fit in ordinary memory. A sweep keeps checkpoints, so an interrupted run picks up where it stopped.

Opening it

- **Serve the folder:** `python -m http.server 8000` in the scAnimator folder, then open `http://localhost:8000`. After `update_dataset_index.py` your dataset has its own card.
- **Or load it straight from disk:** on the landing page, `Load Folder` and pick the dataset folder. The files are copied into the browser's own storage (IndexedDB) and stay on your computer. This works on the live demo too.

WHEN SOMETHING'S OFF

Troubleshooting

Most problems are the browser refusing local files, or a look that suits a different number of cells.

WHAT YOU SEE	WHY, AND WHAT TO DO
Stuck on "Loading data"	The page was opened as a file (`file://`): browsers block `fetch` there. Serve the folder with `python -m http.server 8000`, or use Load Folder.
"No datasets found"	`datasets/index.json` is missing or empty. Run `python scripts/update_dataset_index.py`.
Black screen, no cloud	WebGL2 is off or blocked. Turn on hardware acceleration in the browser's settings; check get.webgl.org/webgl2.
Tiny dots	Point size is in perspective: zoom in, or raise Point Size (2 is the minimum).
Everything white	Too much light: lower Point Intensity and Bloom. Large datasets need much less of both.
Gene not found	Symbols are case-sensitive and must match the dataset (human `CD3E`, mouse `Cd3e`). With `--hvg-only` only variable genes exist.
Gene overlay stays empty	A server without HTTP Range support: the viewer then fetches the whole `expression.bin`, which can take a while for large data. `http.server` and GitHub Pages both support Range.
Slow, low fps	Depth of field, tracers and fractal cost the most. Turn them off while exploring; recording speed doesn't depend on fps.
Recording ZIP never appears	Very long 4K recordings take time to assemble after the last frame (`Finalizing…`). If it reads `ZIP FAILED` , record fewer frames or a lower resolution.
Audio input does nothing	The browser needs microphone permission, and only grants it on `https://` or `localhost`.
Storyboard refused	The message names the step and the problem (no column, value or gene of that name) and lists what exists.

Dataset files

FILE	CONTENTS
`frames.bin`	Float32, frames × cells × 3: every animation frame's positions
`colors.bin`	Float32, cells × 3: the default palette
`metadata.json`	cell count, frame count, n_neighbors label per frame, metadata columns and palettes, gene names
`expression.bin`	Uint8 expression per gene (0–255 of its own maximum), genes one after another
`gene_index.json`	gene → [byte offset, length] in expression.bin, for HTTP Range requests
`gene_ranks.json`	gene → rank by total expression (optional; orders the autocomplete)

The PBMC 3k demo data is “3k PBMCs from a Healthy Donor” by 10x Genomics, licensed under CC BY 4.0, filtered, annotated and re-embedded for this viewer. scAnimator is MIT licensed. Source: github.com/GSNautilus/scAnimator.